

PhotoFlex 产品行为数据远程方案

给 MVP 试用阶段的简单落地指南

目标：让你在远程看到用户如何使用 PhotoFlex，从而判断产品哪里卡住、哪里有价值，同时不把用户的项目文件和原始照片上传到服务器。

一、先说结论

推荐采用“本地项目数据 + 远程行为数据”的组合：项目和照片继续保存在用户电脑的浏览器里；每当用户完成一个重要动作，前端只发送一条很小的匿名事件到 Supabase。你随后可以在 Supabase 中查询数据，或接一个简单的管理页面查看漏斗。

这样最适合当前 MVP：改动小、成本低、容易撤回，也不会因为加统计功能而重做整个产品。

二、目前 MVP 是什么架构？

当前项目是 React + TypeScript + Vite 的前端应用。简单说，Page.tsx、组件和业务模块都运行在用户浏览器中；项目数据主要通过 IndexedDB 保存在本机。它目前没有一个传统意义上长期运行的后端服务器。

部分	现在在哪里运行	它负责什么
页面与交互	用户浏览器	显示页面、响应点击、切换页面
项目与照片索引	用户浏览器本地	保存项目、照片引用、序列草稿
产品行为统计（新增）	浏览器 → 远程接口	记录用户完成了哪些动作

三、推荐的远程数据流

用户操作页面

├ 项目数据：继续写入浏览器 IndexedDB

└ 行为事件：HTTPS → Supabase Edge Function → Postgres → 你的查询/看板

这里的关键点是分开两种数据：项目数据是用户的工作内容，行为事件只是“用户做了什么”的记录。第一阶段不要上传原始照片，也不要上传本地绝对路径。

四、第一版应该记录哪些行为？

事件名	什么时候记录	你能回答的问题
app_opened	打开应用	有多少人真的开始使用？
project_created	新建项目成功	用户能否顺利开始？
folder_added	加入照片目录成功	用户是否走到了核心流程？
scan_failed	扫描失败	哪一步最容易出错？
photo_selected	选择一张照片	用户是否找到了想要的照片？
photo_added_to_table	照片加入工作表	核心动作是否完成？
sequence_created	创建序列	用户是否理解序列功能？
sequence_saved	保存序列成功	用户是否完成了一次有价值的产出？
save_failed	保存失败	是否有数据丢失风险？

建议每条事件都带上少量通用字段：event_name、anonymous_user_id、session_id、app_version、route、occurred_at，以及必要的数量字段（例如照片数量）。

```
{
  "event_name": "sequence_saved",
  "anonymous_user_id": "随机生成的 ID",
  "session_id": "本次打开应用的 ID",
  "app_version": "0.1.0",
  "route": "/sequence",
  "photo_count": 24,
  "occurred_at": "2026-09-04T10:30:00Z"
}
```

五、明确不要收集什么

- 不要上传原始照片、缩略图或照片内容。
- 不要上传本地绝对路径，例如 C:\Users\某人\照片。
- 不要默认上传项目名称、客户姓名、备注等可能包含隐私的文字。
- 不要把事件统计当成用户数据备份；事件丢失时，产品也应该能正常使用。

六、前端需要怎么改？

为了让代码容易理解，可以把统计功能做成一个很小的接口。页面不直接调用 Supabase，而是只调用 analytics.track。这样以后更换服务时，页面代码基本不用动。

```
// src/app/AppDependencies.ts
export type Analytics = {
  track: (eventName: string, properties?: Record<string, unknown>) => void;
};

// 页面或业务模块中
analytics.track('sequence_saved', { photo_count: photos.length });
```

建议先做两个实现：LocalAnalytics（开发时只在控制台打印）和 RemoteAnalytics（正式试用时发送到远程接口）。远程发送失败时只记录错误，不阻塞用户保存序列。

七、Supabase 端需要什么？

Supabase 可以同时提供数据库和一个很小的服务器函数。服务器函数的作用是接收浏览器事件、做基本校验，再写入数据库。这样数据库密钥不会直接放在前端代码里。

```
-- product_events 表（示意）
create table product_events (
  id bigint generated always as identity primary key,
  event_name text not null,
  anonymous_user_id text not null,
  session_id text,
  app_version text,
  route text,
  properties jsonb,
  occurred_at timestamptz not null default now()
);
```

第一阶段只需要一个表和一个 track 接口。等你确认产品真的需要更多数据，再增加登录、权限、团队看板等功能。

八、远程看到数据后，重点看什么？

看板/问题	最简单的计算方式	如果数字很低，先检查
启动率	app_opened 人数	用户是否能成功打开应用
开始率	project_created ÷ app_opened	新建项目入口是否清楚
核心流程完成率	photo_added_to_table ÷ folder_added	扫描、筛选、加入工作表是否卡住
产出率	sequence_saved ÷ project_created	用户是否理解序列，以及保存是否稳定
失败率	失败事件 ÷ 对应成功事件	错误提示、权限和文件兼容性

MVP 不需要一开始就做复杂图表。你甚至可以先在 Supabase 的表格查询里按日期和 event_name 分组，先回答三个问题：用户有没有走到核心流程？在哪一步大量退出？失败是否集中在某一种文件或操作？

九、建议的实现顺序

- 先生成 anonymous_user_id 和 session_id，并确认它们不会包含姓名或路径。
- 先接 LocalAnalytics，在本地操作时确认事件名称和字段正确。
- 在 Supabase 建 product_events 表和 track 接口。
- 接 RemoteAnalytics；网络失败时静默降级，不影响原有功能。
- 用自己的电脑完成一遍流程，检查远程表里是否出现正确事件。
- 邀请 3-5 位试用者，先观察一周，再决定是否需要更多事件。

十、服务器方案怎么选？

方案	适合谁	优点	代价
Supabase（推荐）	想快速做 MVP 的个人开发者	数据库、接口、查询页面集中；上手快	需要学习一点 SQL 和函数
Cloudflare Workers + D1	愿意自己搭更多基础设施的人	轻量、全球边缘运行	概念更多，调试链路更长
Vercel Functions + 数据库	前端已经部署在 Vercel	部署体验顺手	数据库仍要另选，服务分散
PostHog 等托管分析	只想尽快看到产品漏斗	事件、漏斗、看板现成	数据结构和费用受平台约束

当前建议：先用 Supabase 做一版最小闭环。如果你只想验证用户路径、不想写查询页面，也可以先选 PostHog 这类托管分析服务。

十一、第一阶段的完成标准

- 用户仍然可以离线打开项目、添加照片、创建和保存序列。
- 你能在远程表里看到每次试用的匿名事件。
- 你能按日期、事件名和版本筛选数据。
- 网络断开时，用户不会因为统计失败而无法工作。
- 事件里没有原始照片、绝对路径或明显的个人隐私。

十二、参考资料

Supabase 数据库: [官方文档](#)

Supabase Edge Functions: [官方文档](#)

Cloudflare D1: [官方文档](#)

Vercel Functions: [官方文档](#)

PostHog 事件: [事件文档](#)